DevFoundryについて

あなたの最初のビルドと展開

サイトマップ

HTMLCSSJavaScriptでゼロから公開まで

初心者向けのワクブック

空のフォルダから始めます。 公開し、改善しけることができるレスポンシブな個人ポトフォリオで仕上げます。

HTML • CSS • JavaScript • Git • GitHub ページ

第1版 | 2026年11月

完成したポトフォリオのダクモド面。

DevFoundryへようこそ

このコスのタスクは、1つのウェブサイトをビルドして公開することです。 各モジュルは、コンテンツ、設計、レスポンシブレイアウト、インタラクション、バジョン履、デプロイメントなどの有用なレイヤを追加します。 コドがスニペットを集するのではなく、あなたのコドが理解されます。

以前のプログラミングは必要ありません。 エディタとGit、モダンなブラウザ、セットアップと公開のためのインタネット接、およびGitHubアカウントをインストルするための限を持つコンピュタが必要です。 タブレットや携電話だけでは、意したコディング環境ではありません。

オリジナルガイドコスは、約23時間の目安となります。 書、タイピング、デバッギング、課題のコンパイルは、通常は長くなります。 いくつかの短いセッションを計し、自分のペスで作業してください。 最終プロジェクトは、時間試ではなく、追加の練習です。

できること

意味のある見出しとセクションで完全なHTMLページを作成します。

再使用可能なCSS、Flexboxおよび格子とそれ式をして下さい。

320pxの携電話ビュポトからいデスクトップにレイアウトを適させます。

JavaScriptでテマのトグル、モバイルメニュ、プロジェクトフィルタを作成します。

有意義なGitコミットを保存し、GitHubリポジトリを公開し、READMEで作業を明します。

GitHub ページで的なウェブサイトを展開し、みのライブ更新を行います。

このワクブックの使い方

レッスンをんで、その例を入力し、移動する前にタスクを完了します。 ファイルを保存し、すべての意味のある更後にブラウザを更新します。 「」をラベル付けした例は、一時的な練習です。

「portfolio」にラベルを付けた例は、作業プロジェクトに所しています。 コドブロックは、ファイルを全に置き換えるために明示的に言う限り、集中したです。

devfoundry-portfolio という名前の 1 つの作業フォルダを使用します。 ZIPは、各主要なステジの最後に比較するための完全なスナップショットを提供します。 コドがれた場合は、連するファイルと一致するチェックポイントを比較し、プロジェクト全をすぐに交換するのではなく、一致するチェックポイントを比較します。 appendixには、完全な最終的なHTML、CSS、JavaScriptが含まれており、編集可能なファイルとして供給されています。

照名 アレックスモガン、Eメルアドレス、計されたプロジェクトコンセプトは、例えばコンテンツです。 公開前に置換します。 あなたは正直な習プロジェクトを使用することができます。 あなたは、クライアントの仕事を明したり、する必要はありません。

コスマップ

4

モジュール 0 はじめる

9

モジュール1 HTML構造を構築

13

モジュール2のCSSはそれをよく見させます

19

モジュール3 答設計

25

モジュール4 JavaScriptがインタラクティブに

30

モジュール5 GitとGitHub

33

モジュール6 ウェブサイトの展開

36

モジュール7の最終的なプロジェクト

39

モジュール8 次の場所

40

コスのダウンロドとクイックリファレンス

42

CSSチトシト

44

チャレンジと回答ガイド

46

トラブルシュティングデスク

47

リソスディレクトリ

48

Appendix 完全なポトフォリオ HTML

51

Appendix B 完全なポトフォリオCSS

54

Appendix C 完全なポトフォリオ JavaScript

習セッションの提案

セッション1:ツルを設定し、HTMLスケルトンを構築します。 セッション2:CSSを適用し、デザインリファレンスを比較します。 セッション3:モバイルレイアウトを修正し、テストします。 セッション4:インタラクションの構築

セッション5:GitHubリポジトリとデプロイを作成します。 セッション6:最終ポトフォリオをパソナライズ、テスト、提出する。

ビルドマイルストン

空のフォルダ → HTML skeleton → スタイルポトフォリオ → レスポンシブポトフォリオ → インタラクティブポトフォリオ → GitHub リポジトリ → ライブウェブサイト。

各プロジェクトには、察可能なパス基準が含まれています。 チェックポイントが失敗した場合は、トラブルシュティングセクションを使用して、次のレイヤを追加する前にチェックを繰り返します。

モジュール 0 はじめる

ゴル:完成したウェブサイトを理解し、信できるワクスペスを用意します。 ガイド付き時間:約10分、ダウンロドとアカウントのセットアップを除く。

レッスン 0.1 私たちが構築しているもの

あなたの完成したポトフォリオには、ナビゲションバ、導入またはヒロセクション、セクション、スキルリスト、3つのプロジェクトカド、連絡先セクション、フッタがあります。 あなたが誰であるかを理解し、あなたの仕事を見て、あなたに連絡する方法を見つけることができるはずです。 小さな面では、コンテンツのスタックが小さくなり、ナビゲションはボタンで開きます。 テマコントロルは、コンテンツを更することなくカラパレットを切り替えます。

1. フルページのデスクトップタゲット。 すべての7ページ領域は1つの連ポトフォリオにしています。

詳細: 導入および第一次行動、書のために大される。

細部: イメジ、タイトル、記述および行先が付いている完全なプロジェクト カド。

照は、名前や色が例に一致するかどうかのテストではなく、階層とレイアウトのタゲットです。 セクション間の明確な導入、繰り返されたカド設計および大なスペスに注意して下さい。 ヒロの下のプロジェクトカドは同じ幅であり、一貫した像の比率を使用します。

今これを行う

自分の名前、ワンセンテンスの紹介、4つのスキル、3つのプロジェクトアイデアとメモのコンタクトメソッドを書いてください。 1つのプロジェクトしか構築されていない場合、計された練習の念として他の人を記述して下さい。

共有しやすいアドレスを選します。

ご理解くださいますようお願い申し上げます。

ポトフォリオが訪問者に何を役立つか? あなたの焦点を明し、あなたの仕事のを示し、次のステップを提供する必要があります。 便利な容のない美しいレイアウトは出ません。

レッスン0.2 ウェブサイトの仕組み

ブラウザは HTML をみ、コンテンツを理解し、CSS を適用してそれをレイアウトし、JavaScript を行してアクションに答します。 誰かがペジをリクエストすると、サバはこれらのファイルを送信します。

ホスティングは、利用可能なファイルを作るサビスです。 ドメインは、サイトを見つけるために使用される名前です。 URLには、プロトコルやペジパスなどの詳細が含まれます。

フロントエンドの開は、ペジを監視し、やり取りします。 バックエンドシステムは、アカウント、データベス、プライベトサバのロジックなどの作業を理します。 私たちのコスプロジェクトは的なフロントエンドのウェブサイトです。データベスやバックエンドプロセスは必要ありません。

1ペジ、3種類の指示

ブラウザはファイルを結合してペジを生成します。

`index.html`

コンテンツと構造

見出し、セクション、リンク

`style.css`

プレゼンテション

色、間隔、レイアウト

`script.js`

行動規範

クリック、態、フィルタリング

ブラウザ

コンテンツのレンダリング、スタイルを適用し、JavaScriptを行します。

2. 同じブラウザは、異なる責任で3つのファイルタイプを結合します。

お問い合わせ

身近なウェブサイトを開き、ブラウザのサイズを更します。 列、間隔、ナビゲションの更を探します。 HTML はコンテンツ、CSS はプレゼンテションを制御し、JavaScript は動作を追加します。 JavaScriptをオフにすると、HTMLやCSSは削除されませんが、一部のサイトはスクリプトに大きく依存しています。

レッスン 0.3 ツルを設定する

Code.visualstudio.com と git-scm.com から Visual Studio コドをインストルします。 現在のChrome、エッジ、Firefox、Safariブラウザを使用します。 GitHub アカウントを github.com で作成し、電子メルを確認します。 これらのサビスは、面を更することができます。公式の照リンクは、このワクブックの最後にリソスディレクトリにあります。

1。 コスZIPを抽出します。 01-starter-filesをコピして、コピのdevfoundry-portfolioの名前を更します。

2。 VSコドでは、File → Open Folderを選し、devfoundry-portfolioを選します。 フォルダを開くと、ファイルパスと端末が簡に管理できます。

3。 index.html、style.css、script.js を記述します。 アセットフォルダにはコスイラストが含まれています。

4。 お使いのブラウザのindex.htmlをファイルマネジャで開きます。 最初に空白になります。 また、VSコドのロカルプレビュ張機能を利用することもできます。このプロジェクトでは張子は必要ありません。

5。 5。 VSコドのタミナル→新しいタミナルを開きます。 以下のチェックを行します。 出力にはバジョン番が含まれている必要があります。

```
git --version
```

Windows では、端末の例を密接に一致させたい場合は Git Bash を使用します。 macOS では、VS コドの組みみ端末を使用します。 インストル後にGitが見つからない場合は、VSコドを再起動します。 タミナルで示されている開いているプロンプト文字を入力しないでください。コマンド自のみです。

チェックポイント1

index.htmlを編集して保存し、ブラウザで再オプンし、Gitを行できます。 ファイル名が .txt で終わらないことを確認します。 可能であれば、別の存のGitリポジトリの外にプロジェクトを保管してください。 コスZIPには.git履はありません。

モジュール1 HTML構造を構築

Goal: 意味のあるコンテンツと、それをスタイリングする前に完全なページを作成します。 所要時間:約25分 チェックポイント: 02-html-complete.

レッスン1.1 最初のHTMLページ

まず第一ページでスタタindex.htmlを置換します。 要素は通常、オプンタグ、コンテンツ、クロズタグを持っています。 html 要素には、ヘッドとボディが含まれています。 頭部は文書情報を保持します。ボディは目に見えるページの容を保持します。 インデントは、ネスティングを表示し、間違いを簡に見つけます。

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
    content="width=device-width, initial-scale=1">
  <title>My Portfolio</title>
</head>
<body>
  <h1>Hello!</h1>
  <p>I am building my first website.</p>
</body>
</html>
```

doctype は、現代の HTML の動作を要求します。 langは文書の言語を記述します。 UTF-8は通常のテキスト文字をサポトしています。 ビュポト宣言では、ペジは、デバイスの際のビュポト幅を使用して、答性のあるCSSの後に重要になります。 タイトルはブラウザタブに表示されます。h1はペジに表示されます。

ビルドとチェック

見出しと段落が現れて保存、リフレッシュ、確認します。 タイトルを更し、再び更新します。ブラウザタブは更する必要があります。 コド自がテキストとして表示される場合は、.html 張子とそれを確認してください。

```
you opened the saved file. Closing tags use a forward slash, such as </p>.
```

レッスン1.2 テキストとコンテンツ

見出しを使用して階層を記述します。, テキストサイズを選するには. 各主要なセクションの1ペジポトフォリオ1メインh1とh2を指定します。 各プロジェクトタイトルにh3を使用します。 パラグラフはp要素です。

いマクの重要性、そして調表示。 br は、セクション間のスペスではなく、テキストの行のブレイクを作成します。CSS は間隔を理します。

ポトフォリオ例:導入後のこのセクションをインサトします。 その後、メインに移動します。

```
<section id="about">
  <h2>About me</h2>
  <p>I'm Alex, a <strong>beginner developer</strong>.</p>
  <p>I enjoy building useful websites and
    <em>learning by doing</em>.</p>
</section>
```

ミニプロジェクトについて

自分自身について2つの短い段落で例を置換します。 あなたが習しているものや、あなたがしむものが含まれています。 意味が正化するときだけいか em を使用してください。 あなたのセクションは、ブラウザがh2を表示し、みやすい段落とあなたのペジはまだ明確なメインh1を持っているときに渡します。

レッスン1.3 リンクと像

アンカ、a、別の場所に移動します。 その href は目的地をえます。 などフラグメントリンク

```
#about moves to the element whose id is about. IDs must be unique within the page. An image
uses
src for its file path and alt for a text alternative. img is a void element: it has no closing
</img> tag.
<a href="https://github.com/">Visit GitHub</a>
<a href="#about">About me</a>
<img src="assets/workspace.svg"
```

幅=″520″ 高さ=″360″

```
  alt="Illustrated code editor with HTML, CSS and JavaScript files">
```

パスは HTML ファイルに相的です。 Assets/workspace.svg は「このペジのにあるアセットフォルダの中を見る」を意味します。 フォワドスラッシュを使用して、正確に資本化を一致させます。 意味のある像は有用なaltテキストを必要とします。純に飾的な像はalt=″を使うことができます。 alt テキストの語のキャプション語を繰り返すことを避けて下さい。

ミニチャレンジ

バンドルされたワクスペスイメジを追加し、GitHubへのリンクとFragmentリンクを〔About〕セクションに追加します。 方のリンクをクリックします。 意的にイメジパスの名前を更し、失敗を察し、それを修正します。 C:\Users\\Alex などのコンピュタからパスを使用しないでください。他の訪問者はアクセスできません。

レッスン1.4 ダイバシティとセマンティックHTML

セマンティック要素は、地域の目的を記述します。 ヘッダは、ペジや領域、 navグループプライマリナビゲション、メインはペジのメインコンテンツ、セクショングループの名前のトピック、記事は、自己完結した項目を保持し、フッタはペジや領域を閉じます。 div は、より有意義な要素が合わないときにレイアウトに役立つニュトラルコンテナです。

HTMLのランドマクとネスティング

セマンティック要素は、各地域の目的を記述します。

ヘッダ

nav: ブランド、セクション リンク、制御

メインページ

セクション#ホムh1の紹介

セクション#about h2について

セクション#skills h2スキル

セクション #projects h2 + 記事

セクション #contact h2 コンタクト

フッタ

名前とバックツトップリンク

プロジェクトの記事

意味のあるプレビュ+テキストalt

h3プロジェクトタイトル

p 正直な記述

利用可能な場合の際の目的地

中立的なレイアウトのラッパのためのdivを使用して下さい。

3。 ランドマクとネストされたプロジェクトコンテンツ。 タグはスタイリングの前に目的をえます。

1つの主要なランドマクを使用して下さい。 ヘッダ、メインの5つのメインコンテンツセクション、およびメイン後のフッタでナビゲションをします。 セクションは通常見出しを必要とします。 プロジェクトカドは、それぞれ立したプロジェクトの要であるため、記事を使用します。 リンクの周りのラッパはdivを維持することができます。それは明された意味を必要としません。

練習とチェック

存のコンテンツをヘッダ、メイン、フッタに再構成します。 本文のすべての表示テキストを保持します。 あなたのページは同一に見えるかもしれません: 意味を意味する意味を設計を自動作成しないで改良して下さい。 #about は、そのセクションを移動した後にまだ見つけていることを確認します。

レッスン1.5 ポトフォリオ構造の構築

以下に示すフルページにメインを大します。 これは構造計です。完全な容は、付Aと 02-html-complete/index.htmlにあります。 比較が必要な場合は、フルチェックポイントのHTMLをコピし、例のテキストを自分で置き換えます。 CSSやJavaScriptをまだ接しないでください。

```
<header><!-- navigation --></header>
<main id="main">
  <section id="home"><h1>Your introduction</h1></section>
  <section id="about"><h2>About</h2></section>
  <section id="skills"><h2>Skills</h2></section>
  <section id="projects"><h2>Projects</h2></section>
  <section id="contact"><h2>Contact</h2></section>
</main>
<footer><!-- name and back-to-top link --></footer>
```

未注文リスト、li アイテムを持つ ul を使用して、スキル。 プロジェクトカド、それぞれイメジ、h3、明の3つの記事要素を作成します。 完成したプロジェクトは、際の目的地のリンクを持つことができます。 プラン

プロジェクトは、誤解を招いたリンクではなく、テキストとして「近日公開」を示すべきです。 完全な照には、モジュール 4 用のれたコントロルも含まれます。 非表示は、これらの非アクティブボタンをページ外に保持して、その動作が存在するまで保持します。

プロジェクト 1 ポトフォリオ スケルトン

配信可能: 1つの完全で、未定のHTMLページ。 あなたの名前と導入が最初に現れ、その後、約、スキル、プロジェクト、連絡先がきます。 フッタを含める。 少なくとも3枚のカドは、計された念を明らかにしても存在しなければなりません。 メトリンクを使用して連絡先; それは訪問者の構成されたメルアプリを開き、それ自でメッセジを送信しません。

すべてのナビゲションリンクが一致するユニクなIDに達すると、各像がみまれたり、意味のある代替品を持っている場合は、見出し順がセンシブルで、コンテンツは外ではありません。 02-html-complete と比較してください。 このステジのプレンページが予想されます。次のモジュルはデザインをえます。

モジュル2のCSSはそれをよく見させます

Goal: 一貫性のあるデスクトップデザインを作成します。 所要時間:約35分 チェックポイント: 03-css-complete. このチェックポイントは、モジュル3で改善できるように、固定デスクトップ幅を使用しています。

2.1 CSSを接する

インサイドヘッドはスタイルシトリンクを追加します。 それからstyle.cssでボディ ルルを入れて下さい。 ファイルを保存してリフレッシュします。

更はスタイルシトが接されていることを明します。

```
<link rel="stylesheet" href="style.css">
body {
背景: #f8fafc; 色: #172033; フォント家族: Arial、Helvetica、sans-serif;

}
```

セレクタは要素を選します。括弧の宣言は値を割りてます。 ボディは要素タイプを選し、.heroはクラスを選し、#projectsはIDを選します。 複の要素はクラスを共有できます。 ID は 1 つの要素を識別します。 class="hero" を .hero にマッチさせるセクションに置きます。

2つのルルが同じプロパティをタゲットにすると、カスケドはどのウィンを獲得するかを決定します。 これらの例では、通常、より特定のセレクタが勝っており、後でスタイルシトの特定のルルは以前のものよりも勝ちます。 スキャタリングではなく、ブラウザのスタイルのパネルで、ほとんどのスタイリングと予期しない結果をチェックするためにクラスを使用してください! 重要な宣言。

ビルドとチェック

スタイルシトを接し、h1色を一時的に更します。 再保存後。 何も更しなければ、style.css のスペルを確認し、ファイルを保存し、ブラウザのネットワクまたはコンソルパネルを査して失敗したリクエストを理します。 コンソルは、ブラウザのメニュやペジを調べて、開者向けツルです。

レッスン2.2 色フォントと

再利用可能な設計値はウェブサイトの一貫性を維持します。 カスタムプロパティは、多くの場合、CSSと呼ばれ、2つのハイフンで始まります。 : root はトップレベルの HTML 要素なので、ドキュメント全で利用可能なが宣言されます。 var() は値をみます。

```
:root {
--bg: #f8fafc; --surface: #ffffffffff; --text: #172033; --muted: #475569; --accent: #1d4ed8; --line: #cbd5e1;

}
body {
背景: var(---bg); 色: var(--text); フォント家族: Arial、Helvetica、sans-serif; フォントサイズ: 1rem; ライン高さ: 1.65;

}
```

背景は、ページキャンバス、表面はカド背景、テキストはメインコピであり、ミュトは二次コピであり、ハイライトリンクとボタンを調し、領域を分離します。 すべての明るい色がテキストとして機能するのではなく、みやすいコントラストを選します。 ブラウザのアクセシビリティツルは、コントラストを測定するのに役立ちます。

練習とチェック

をボディ、リンクおよびプロジェクト カドに適用して下さい。 --accent だけ更し、それを使用してすべてのコンポネントが一に更新することを確認します。 ボディラインの間隔を保ち、限られたのテキストサイズ。 Appendix B にフルパレットと型スケルが現れます。

2.3 CSS ボックスモデル

各要素は箱を占めます。 コンテンツはテキストやイメジです。 パディングは、境界線の中にスペスを作成します。

マジンは、境界線の外側にスペスを作成します。 Border-radiusは角を丸くしますが、スペスを追加しません。

箱の目に見える次元は箱サイジングによって決まります。

CSSボックスモデル

外部から部へ:マジン、ボダ、パディング、コンテンツ。

マジン

パディング

コンテンツ

ボダ:1px

ボダボックスの幅

300px合計

各側面にパッドを入れる20px

1px の境界線

258px コンテンツ

マジンは外に座っています。

4. ボダボックスでは、宣言された幅には、コンテンツ、パディング、ボダが含まれますが、マジンは外部にとどまります。

```
* { box-sizing: border-box; }
.project-card {
幅:300px;パディング:20px; ボダ:1pxソリッドバ(--line); ボダラディウス:12px; 金:20px;

}
```

ボダボックスでは、幅300pxのパディングとボダが組みまれています。容の幅は258pxです。 デフォルトのコンテンツボックスでは、ボダボックスは342px幅です。 マジンは方の測定の外側に座っています。 完了したGridレイアウトでは、この固定されたカド幅を削除して、列が制御します。最終的なカドは、代わりに.card-bodyでパディングを使用します。

ミニチャレンジプロジェクトカド

上記の規則で1つの記事を一時的にスタイルします。 開者ツルでは、ボックスモデルを調べ、4つのレイヤを識別します。 パディングの加と更の記述 最後の .project-card と .card-body ルルを Appendix B から適用する前に、一時的な幅とパディングを削除します。

レッスン2.4 Flexbox

Flexboxは、コンテナの直接の子供をメインの軸に沿って配置します。 定の flex-direction: 行で、主軸は水平方向に行し、方向に交差軸線を移動します。 列の更と軸の回。 正化コンテンツは、メインの軸線に沿って整列します。 ギャップを外さないで子供を分離します。

Flexboxは主要な軸線および十字軸を持っています

flex-direction: 行で表示します。 列に切り替えて軸が回します。

表示: flex; ギャップ: 24px; ブランド リンク ボタン 主な軸: 正性 十字軸: 整列項目 5. 列ベスのFlexboxコンテナの主軸と交差軸のアライメント。

```
.nav {
ディスプレイ:フレックス;アライメント:中心;ギャップ:1.25rem;

}
.brand { margin-right: auto; }
.nav-links { display: flex; gap: 1.25rem; }
.hero { display: flex; align-items: center; gap: 3rem; }
.hero > div { flex: 1; }
.hero-art { width: 42%; height: auto; }
```

ブランドの自動右側のマジンは、スペア水平空間を吸します。 ヒロのテキストラッパは、利用可能なスペスを使用するために成長します。像は行の一部を占めます。 これらのルルは、Appendix Aからクラス名を定します。直接の子供だけがフレックスアイテムになります。ネストされた段落は、自動的に.heroのフレックスアイテムではありません。

ビルドとチェック

これらのルルをナビゲションとヒロに適用します。 ディスプレイを追加: flex, flex-wrap: .skills へのラップとギャップなので、バッジはラップできます。 flex-direction: 列を一時的に試し、軸線の言語が「方向アライメント」よりも正確である理由を察してください。 デスクトップヒロの行を今復元します。

レッスン2.5 CSSグリッド

グリッドは行と列を一に制御します。 繰り返されたプロジェクトカドのためにそれを使用して下さい。 各記事は.projects-gridの直接子です。 摩擦、fr、ギャップの後の利用できるトラック スペスを共有して下さい。 minmax(0, 1fr)は、意したよりもはるかにいトラックを制することなく、列の縮小を可能にします。

CSS Gridは行と列を渡るスペスを共有します

繰り返し(3,minmax(0, 1fr)) は、3つの同じようにフレキシブルなトラックを作成します。

カド 1

カド2

カド3

カド 4

カド 5

カド 6

1frの

1frの

1frの

トラックを分けます。外側のパディングではありません。

6. 格子トラックは利用できるスペスを共有します。 4番目のカドは新しい行に流れます。

```
.projects-grid {
表示: グリッド; グリッドテンプレトコロン: 繰り返し(3, minmax(0, 1fr); ギャップ: 1.5rem;

}
.project-card {
オバフロ: されている; 背景: var(--surface); 境界: 1px 固var(--line); 境界線: .9rem;

}
.card-body { padding: 1.4rem; }
.project-card img { width: 100%; height: auto; }
```

ビルドとチェック

3つのプロジェクト記事を div に class project-grid に入れます。 ルルを適用し、デスクトップ照の幅で3つの等しい列を確認します。 として繰り返す(3、...)を繰り返します(2、...)。 3番目のカドは別の行を開始する必要があります。 3つの列を復元します。 これらのデスクトップの列は、電話にあまりにもタイトになります。これは、特定の問題モジュル3は解決します。

レッスン2.6ホバ果とトランジション

ホバ態はマウスユザのフィドバックをえます。 目に見える焦点の態はキボドのユザを助けます。
トランジションは値間の化を滑らかにし、隣接するカドを切らずに視的にカドを移動させます。 動きを小さく、意的に保ちます。

```
.project-card { transition: transform .2s ease; }
.project-card:hover,
.project-card:focus-within { transform: translateY(-4px); }
:focus-visible {
輪郭: 3px固var (----accent);輪郭オフセット: 4px;

}
@media (prefers-reduced-motion: reduce) {
  .project-card { transition: none; }
  .project-card:hover,
  .project-card:focus-within { transform: none; }
}
```

ビルドとチェック

カドをハバし、タブを使用してリンクを集中します。 方の相互作用はフィドバックを提供するべきです。 焦点の輪郭を取除いないで下さい。 タッチデバイスにはホバがないため、必須情報は必要ありません。 減少した感情の設定は、最終的なスタイルシトでこの動きをオフにします。

レッスン2.7 ウェブサイトのデザインを完了

Appendix B で完全なベスCSSを使用して、「3.2 Mobile が最初にオバライドしてブレイクポイント」コメントの前に停止します。 これは、03-css-complete/style.cssと同じ容です。 テマとタイプ、共有要素、Flexbox、Grid、フィドバックを追加する前に各グルプをんでください。 同じクラスを象とする一時的な練習ルルを置換します。

ベス設計では、1100px .container をマジンインラインでセンタリングしています。 セクション共有パディング。 ボタンとリンクはアクセントカラを繰り返します。 プロジェクトの像は同じアスペクト比を使用しており、カドテキストは自然に育つことができます。 高さは段落の周りに固定されていないので、長いテキストはクリップされません。

プロジェクト2 ポトフォリオ再設計

配信可能: スタイルのデスクトップポトフォリオ。 1200px以上のブラウザでテストします。 照からフルページをチェックしてください:みやすいタイプ、力なヒロ、等しいカド列、有用なコントラスト、一貫性のある境界線、丸みのある角、および明らかなリンク。 キボドに焦点を合わせると、スキップリンクが表示されます。

設計がヘッダからフッタに一貫した感じのとき、イメジは歪められず、キボドの焦点は見られます渡して下さい。 水平方向のオバフロをショトカットとしてさないでください。 堅い容器を取り替え、モジュル3のレイアウトをえて下さい。

モジュール3 答設計

目的:あなたの設計は利用できるスペスに滑らかに合わせます。 所要時間:約20分

チェックポイント: 04-レスポンシブ完了。

3.1 なぜウェブサイトが電話にかかっているのか

その前提がもはや合わないときデスクトップのレイアウトはれます。 1100pxのコンテナは390pxのビュポトをオバフロします。 3つのいカドはテキストをみにくいようにします。 長いナビゲション列は、面をオフ制御することができます。 レスポンシブデザインは、ペジ全を縮小するのではなく、それらの制約を更します。

7. デスクトップ、タブレット、および電話幅でスクリンショットを一致させます。 デバイス名ではなく、スペスでレイアウト更。 デスクトップの要。

768pxのタブレット。 同じページの上部のクロップを大しました。

390pxの携電話。 同じページの上部のクロップを大しました。

開者ツルを開き、レスポンシブデバイスツルバを有にし、ビュポト幅を削減します。 フィットしない最初の要素を見つけます。 CSSを更する前に、計算された幅とその親を調べます。 通常のブラウザウィンドウのサイズを更することもできます。 プリセットデバイスフレムは便利ですが、すべての中間サイズをテストしません。

試してみる

390pxでは、デスクトップチェックポイントで水平スクロルの1つのソスを特定します。 それは固定されるべきです

```
.container. Write down the cause before applying the next lesson's fix. Do not use overflow-x:
hidden
```

問題をすために身に:それはアクセスできないコンテンツをすことができます。

レッスン3.2 メディアの問い合わせ

最小のレイアウトで始まり、部屋があるときに列を追加します。 デスクトップベスCSSの後、または04-response-complete/style.css全を使用するこれらの規則を追加します。 初期のデスクトップ値が上書きされると、同等に固有のルルです。

```
.container { width: min(1100px, calc(100% - 2rem)); }
img { max-width: 100%; height: auto; }
.projects-grid { grid-template-columns: 1fr; }
@media (min-width: 640px) {
  .projects-grid {
```

格子templateコラム:繰り返す(2、minmax (0、1fr));

```
  }
}
@media (min-width: 960px) {
  .projects-grid {
```

格子templateコラム:繰り返し(3、minmax (0、1fr));

```
  }
}
```

コンテナは、最大幅に達するまで、各側面に1つのスペスを保持します。 件がの場合、メディアクエリが適用されます。特定のデバイスモデルを知しません。 弊社のブレクポイントは、640px から 959px までの 1 つの列で、960px 以上の 3 つを設計します。

コンテンツが異なる幅を必要とする場合は、後で更してください。

チェックイン

639、640、959、960のCSSピクセルでテストします。 列の更は、宣言された境界線で、欠落しないカドで起こるべきです。 デバイスのツルバにCSSピクセルを使用します。携電話の物理的な面ピクセルは異なる測定です。

レッスン 3.3 答ユニット

px は CSS ピクセルで、境界線やブレクポイントによく機能します。 rem は root フォントのサイズに相的です。1rem は一般的に 16px のブラウザのデフォルトでは、訪問者は更できます。 パセンテジはプロパティに依存し、コンテキストを含む。 vwはビュポト幅の1パセントです。vhはビュポトの高さの1パセントです。 全高さのモバイルヒロを想定しないように、常にブラウザのコントロルに適します。

```
h1 { font-size: clamp(2.25rem, 5vw, 3.5rem); }
.hero { flex-direction: column; gap: 2rem; }
.hero-art { width: min(100%, 520px); }
@media (min-width: 960px) {
  .hero { flex-direction: row; align-items: center; }
  .hero-art { width: 42%; }
}
```

lock() は最小値と最大値の柔軟な値を選します。 ヘッドラインが小さくても大きくなってしまうのを防ぎます。width: min(...) は、縮小する時にイメジを制限する、より小さい値を選します。 付は、フッタレイアウトを含むすべての答オバライドを含みます。

練習とチェック

ブラウザのテキストサイズまたはズムを200%に加させます。 見出しがまだラップされていることを確認してください。ボディテキストはみやすく、固定高さでクリップされていないことを確認してください。 各項目を密な座標で置くために試みる上で適用範がいサイジングおよび自然な文書の流れを好みます。

レッスン3.4 モバイルナビゲション

JavaScriptの前に、すべてのナビゲションリンクが表示されるようにします。 Flex-wrap はデスクトップのナビゲションをラップすることを可能にします。

小さな面では、リンクが列になります。 メニュボタンは、そのクリック動作がモジュル4に存在するまでされます。 このベジは、まだ作成中です。

```
.nav { flex-wrap: wrap; }
@media (max-width: 639px) {
  .nav-links {
```

幅: 100%; flex 方向: 列; 順序: 3;

```
  }
  .nav-links a { padding-block: .5rem; }
  .js .nav-links { display: none; }
  .js .nav-links.is-open { display: flex; }
}
```

.js クラスは、完了したメニュスクリプトでのみhtmlに追加されます。 手動で追加しないでください。 大面では、小さな面の非表示ルルは適用されません。 終了メニュは、通常のナビゲションリンクを制御する通常のボタンです。アプリケションスタイルのメニュロルを必要としません。

チェックイン

320pxでは、すべてのリンクが表示され、みやすく、アクティベションが容易です。 デスクトップの幅で、リンクは行を形成します。

```
Later, the button's aria-expanded state must match whether those links are open on small
screens.
```

レッスン3.5 ウェブサイトをテストする

テストは、最終的な一目ではなく、繰り返す活動です。 以下の行列を使用して、課題/test-log.md で問題を記します。 最小面で起動し、上に移動します。また、サイズ間のウィンドウをドラッグします。

テスト

期待される結果

320px 電話

1列、みやすい見出し、水平ページスクロルなし

390px 電話

快適な間隔および使用可能な運行

768pxタブレット

2つのプロジェクト列と積み重ねられた英雄

1440pxデスクトップ

3つのプロジェクト列、イメジのにあるヒロテキスト

200%ズム

テキストとコントロルはクリップなしで使用可能です

テスト

期待される結果

キボードのみ

すべてのリンクが到達可能でフォカスが見える

プロジェクト3 レスポンシブポトフォリオ

配信可能: 320px からデスクトップで使用可能な一ページ。 Appendix B で完全なレスポンシブブロックを追加し、04-レスポンシブコンと比較してください。 箱型から左に固定されたカド幅はありません。

像が比例して縮小するときに、小さな面にヒロとフッタのスタック、プロジェクトは予想される列カウントを使用して、ナビゲションはアクセス可能であり、水平なオバフロはありません。 各タゲットレイアウトから1つのスクリンショットを自のテストノトに含めます。 利用可能な場合、際の電話テストは便利です。ブラウザのプレビュはまだ必須のベスラインです。

モジュール4 JavaScriptがインタラクティブに

目的: コンテンツを使用可能に保ちながら、制御は予測可能に反させます。 ガイド時間:約25分、プロジェクト4ラボ。 チェックポイント: 05-javascript-complete.

レッスン4.1 JavaScriptとは

HTMLは制御とコンテンツを提供します。 CSS は可視態を定義します。 JavaScriptは何かが起こるとき態をえます。 テマのトグルのために、クリックして上のクラスを更し、CSS

```
variables change the palette. For the mobile menu, a click changes whether the navigation
links are
```

オプン。 フィルタリングのために、プロジェクトカドがされているクリック更。

ナビゲションとボタンを操作するためのリンクを保持します。 クリックハンドラを持つdivは、ネイティブボタンが動作するキボドを自動的にサポトしません。 JavaScriptが利用できなくなった場合、ペジのコンテンツとリンクは使用できません。設定されたコドが行した後にのみ化されたコントロルが公開されます。

コディング前の計

各機能について、アクション、態、目に見える結果について明します。 例:「ダクモド→ダククリック」はの→ボディはクラス暗く、ボタンのレポトが押された」 正確な文は、コドをテストしやすくなります。

レッスン 4.2 最初のJavaScript

stylesheet リンクの後に、このスクリプト要素を頭の中に追加します。 HTML の解析をブロックせずにスクリプトをダウンロドし、ドキュメントが解析された後に行しますので、要素は選できます。

```
<script src="script.js" defer></script>
```

このでscript.js に一時的なコメントを置換します: console.log("Hello DevFoundry!");; 保存してリフレッシュ. ブラウザの開者コンソルを開き、メッセジを見つけます。 表示されない場合は、script.js が正しくスペルされ、失敗したファイルリクエストをチェックします。 シンタックスエラは、行からスクリプトを停止することができます。コドを追加する前に、最初のエラと行番をみます。

チェックイン

メッセジを更し、保存し、再び更新します。 新しいテキストが表示されます。 完了したプロジェクトスクリプトを追加する前に、このを削除します。 コンソルメッセジは行を明しますが、可視的なユザインタラクションではありません。

レッスン4.3と

```
const creates a binding you will not reassign. let allows reassignment. Strings hold text,
numbers
```

arithmetic をサポトし、 booleans は true または false を表します。 グルプの明と、パラメタと呼ばれる入力を受け入れることができます。 の定義は行しません。括弧で呼び出します。

```
const name = "Alex";
let projectsBuilt = 1;
function sayHello(person) {
  return `Hello ${person}`;
}
```

console.log(sayHello(name)); projectBuilt = projectBuilt + 1; console.log(projectsBuilt);

```
Template literals use backticks and ${...} to insert a value into text. return sends a value
back to the
```

呼び出し者。 パラメタは、指定した入力のロカル名です。 テマでは、ボオランはダククラスが存在するかどうかを決定します。

ミニチャレンジ

"I build " を返す記述Project(title) をタイトルにけて記述します。 お問い合わせ

```
portfolio's title and log the result. Expected answer: return `I built ${title}`. Replace this
experiment
```

その後、際の機能コドを使ってください。

レッスン4.4 DOMの基本

ドキュメントオブジェクトモデルは、ペジのブラウザのオブジェクト表現です。

```
document.querySelector finds the first matching element using a CSS-style selector. It returns
null if
```

何もマッチしません。 classList はクラスを更し、textContent はプレンテキストを更し、setAttribute は性を更新します。

```
const button = document.querySelector("#theme-button");
```

console.log(button); モジュル1からの完全なポトフォリオHTMLでこれを行します。 ボタン要素は null ではなく表示されます。

リド # は ID を選します。 主要なドットはクラスを選します。 スペルは正確に一致しなければなりません。 前のレッスンの defer 性は、この選が HTML が解析される前に行されることを防ぎます。

チェックイン

button.textContent = "テマ" を一時的に使用し、リフレッシュします。 選されたボタンは機能が準備されるまでされますが、開者ツルでそれを査すると更されたテキストが表示されます。 その後「ダクモド」を復元します。 コンソルがnullのプロパティをみ取りできない場合は、イベントコドを更する前にセレクタとHTMLを確認してください。

レッスン4.5イベント

```
addEventListener connects an event to a function. The function runs later, when the event
happens.
```

矢印は、その機能を供給するためのコンパクトな方法です。 り値を使用するという意味のない限り、ハンドラを呼び出しないでください。

一時的な:存のテマボタンを明らかにし、各クリックを記します。 際の機能をインストルする前に、このを削除して、ワンクリックで重複したハンドラを行しません。

```
const button = document.querySelector("#theme-button");
button.hidden = ;

button.addEventListener("click", () => {
console.log("ボタンをクリック!");

});
```

チェックイン

ボタンを2回クリックし、2つのメッセージを確認します。 タブでリチし、 Enter または Space でアクティブにします。 ネイティブボタンはこれを自動的にサポトします。 ハンドラがコンソルにのみ書きむため、目に見えるページはまだ更されていません。 次に、ハンドラはクラスとアクセシビリティの態を更新します。

レッスン4.6 ダクモドとライトモドの構築

すでに完全なスタイルシトを使用していない場合は、Appendix Bからbody.darkオバライドを追加します。
まず、この最小バジョンをきれいなscript.js でします。 「ダクモド」をラベル付けしたボタンを使用します。
押された手段はダクモドが有です。

```
const themeButton = document.querySelector("#theme-button");
テマButton.hidden = false;

themeButton.addEventListener("click", () => {
  const isDark = !document.body.classList.contains("dark");
  document.body.classList.toggle("dark", isDark);
  themeButton.setAttribute("aria-pressed", String(isDark));
});
```

8. 光と暗いテマのイデンシャルコンテンツ。 パレットとテマの態が化するだけ。 ライトテマ。

ダクテマ。 同じコンポネントはレイアウトとアクセス可能な制御を保持します。

お問い合わせ 演算子は boolean を逆させます。 classList.toggle(name, true) はクラスを追加し、 false は削除します。

```
aria-pressed tells assistive technology the current state; styling alone does not do that.
String()
```

booleanを性テキスト「true」または「false」に換します。

永性を追加

Appendix C の完全版は localStorage から保存されたテマをみます。それ以外の場合は、色調のスキムを使用します。 setTheme はクラスとプレス態を一に保持します。 クリックすると、新しい選肢が保存されます。 一部のブラウザの設定が防止できるため、ストレジアクセスは試/キャッシュでラップされます。 ストレジが失敗した場合は、現在の訪問にはまだ機能します。 このバジョンでは、ペジのみ時にシステムの設定をチェックします。ライブオペレティングシステムのテマの更は、訪問者が選を保存していない限り、次の負荷に果をもたらします。

次のラボの前にAppendix Cで最小スクリプト全を置き換えます。 方のバジョンを一に貼り付けないでください。コンジット宣言を複製することは、構文エラです。

プロジェクト4 インタラクティブポトフォリオラボ

このプロジェクトは、テマのトグル、インタラクティブなモバイルナビゲションと1つの追加の相互作用の元の3つの相互作用の要件を完了します。 追加の機能は、プロジェクトフィルタリングです。 3つのブロックでAppendix Cでフルスクリプトをみ、05-javascript-completeと比較します。

ラボ A モバイルナビゲション

```
The HTML button uses aria-controls="nav-links" and aria-expanded="false". The script selects
the
button and link wrapper. setMenu changes .is-open and aria-expanded together. syncMenu closes
```

メニュは640px未のボタンをクリアします。 メディアケリチェンジリスナは、ビュボトが境界線を渡したときにレイアウトを同期させけます。

メニュを切り替える ナビゲションリンクを選すると、小さな面に閉じます。 エスケプはオプンメニュを閉じ、そのボタンにフォカスを返します。 このコドは、 .js を html に追加します。 スクリプトが無な場合、リンクは表示されません。

ラボBプロジェクトフィルタ

```
The HTML buttons have data-filter values all, web and javascript. Each project article has a
matching
data-category. dataset reads these custom attributes. querySelectorAll returns a collection;
the
```

スプレッド構文〔...〕は配列に換します。 各項目のを行します。 コドは非表示

```
nonmatching cards, updates aria-pressed on every filter and announces the visible count through
```

ロル="status"

```
// Core decision inside filterProjects(category)
```

Card.hidden=カテゴリ! ==「すべて」&&&

カド.dataset.category！==カテゴリ;！=== は“等しくない”を意味し、 && は方の件がでなければなりません。 すべての選で、最初に

```
condition is false so no card is hidden. The CSS rule [hidden] { display: none !important; }
ensures
```

コンポネントのレイアウトルルは、れたカドを誤って表示しません。

テストおよびパス

テマボタンを2回クリックします。パレットとプレス態が一にわります。

テマを選し、同じ起源でリフレッシュし、保存時に選したままにしてください

利用できる。

390px で、メニュを開き、Escape を選します。 メニュは回閉じます。エスケプ

```
 returns focus to Menu.
```

モバイルからデスクトップ、バックまでのサイズ更 ナビゲションは使用可能であり、モバイルメニュはリセットされます

休館日

Webを選:2枚のカドショ。 JavaScript を選: 1 つのショ. すべてを選: 3つのショ.

JavaScriptを無化し、リフレッシュ:すべてのナビゲションリンクとカドが表示されるようになりました。

すべてのチェックが成功し、コンソルが予期しないエラがないときに渡します。 コントロルが不在の場合、フルスクリプトが接されていることを確認し、以前のエラはありません。 HTML に .js クラスを手動で追加しない。

モジュール5 GitとGitHub

Goal: 便利なプロジェクト履を作成し、明確なリポジトリを共有します。 所要時間:約20分

作業フォルダにけてください。もちろんバンドルのルトではありません。

Lesson 5.1 開者がGitを使用する理由

Git は、リポジトリで選した更のスナップショットを記します。 コミットは、メッセジとアイデンティティをスナップショットします。 史は、いつ、なぜ何かがわったのか理解するのに役立ちます。 ロカルでGitを行します。

GitHubはリポジトリをオンラインでホストし、コラボレションツルを追加します。別のサビスです。

Git履は、プッシュする前にロカルです

ファイルの保存、コミット、デプロイは別のアクションです。

ワキングツリ

ファイルの編集

ステジングエリア

更の選

ロカルコミット

履を保存

GitHubリモト

コミットを共有する

```
git add
git commit
git push
```

保存前の査

```
git status   |   git diff   |   git diff --cached
```

9。 リモトで共有する前に、ロカルで更を選してコミットします。

作業ツリには、編集したファイルが含まれています。 ステジング領域は、次のコミットのために選した各更のバジョンを保持します。 コミットは、これらのステジされた更をロカルに保存します。 プッシュがコミットをリモトリポジトリに送信します。 ファイルを保存してもコミットしないで、コミットはライブウェブサイトを公開しない。

トレーニング

自の語でこのシケンスを記述: 編集 → チェック → ステジ → コミット → プッシュ. それをステジングした後にファイルを再び編集すると、新しい更を再びステジします。 git ステタスは、段階的な更とステジが解除された更を別するのに役立ちます。

レッスン5.2 Gitリポジトリを作成する

devfoundry-portfolio で VS コドタミナルを開きます。 端末に表示されているフォルダは、コマンドを行する前にプロジェクトです。 名前を構成し、電子メルを一度コミットして下さい。 例の値を置き換えます。プライバシのために、アカウントのメル設定から、GitHubで承認されたnoreplyアドレスを使用できます。

```
git config --global user.name "Your Name"
git config --global user.email "YOUR_COMMIT_EMAIL"
git init
git branch -M main
git status
```

プロジェクトのルトで .gitignore を以下の容で作成します。 主要なドットはファイル名の一部です。 これは、Git によってに追跡されていないファイルではなく、一般的なロカルの混を除外します。

```
.DS_Store
```

サムネイル

```
.vscode/
```

ステジング前の更点を調べます。 git diff は未段階のテキスト編集を示しています。新しい追跡されていないファイルはまだ開いてレビュする必要があります。 プロジェクトをステジにし、ステジされたサマリとコミットを調べます。

```
git add .
git diff --cached --stat
git commit -m "Build responsive interactive portfolio"
git status
git log --oneline -3
```

チェックイン

1つのコミットときれいな作業ツリが表示されます。 Git が「Author 本人不明」と言った場合、名前とメルをセットし、コミットを再試行します。 「コミットしない」と言うと、あなたはすでに日付または間違ったフォルダにあるかもしれません。 パスワド、トクン、またはプライベト連絡先デタを公開しないでください。パブリックポトフォリオリポジトリは他の人に表示されています。

5.3 GitHubにプロジェクトをアップロドする

GitHub では、devfoundry-portfolio というリポジトリを作成します。 このコスの無料GitHubペジワクフロで公開可視性を使用します。 すでにロカルリポジトリを持っているので、 README、 .gitignore およびライセンスの初期化を解除して、リモトが空の起動します。 HTTPS の URL GitHub のショをコピします。

以下のコマンドで、あなたのアカウント名をアカウント名に置き換えます。 ロカルプロジェクトフォルダで行します。 プレスホルダアングルブラケットは際のコマンドに含めないでください。

```
git remote add origin https://github.com/YOUR_USERNAME/devfoundry-portfolio.git
git remote -v
git push -u origin main
```

GitHub がサポトしている HTTPS 認フロを使用する際の認 通常のアカウントパスワドは、Git HTTPS 操作のパスワドではありません。 Git Credential Manager または GitHub CLI は、ブラウザのサインインフロを提供できます。 正式なセットアップリファレンスを照してください。 トクンをコミットしたファイルに入れたり、スクリンショットで共有したりしないでください。

チェックと回復

リポジトリペジを更新します。 index.html、style.css、script.js、およびアセットはルトに表示されるはずです。元の存在がすでに存在する場合は、gitリモト-v を調べます。 git リモトの set-url の原点と間違った URL を正しい

URL で修正します。 リモートがREADMEコミットを持っているので、プッシュが拒否された場合、制プッシュしないでください。 この最初の演習のために空の新しいリポジトリを使用して、または履を再確認するのに役立ちます。

レッスン5.4 プロフェッショナルなREADMEを作成する

README.md explains what the project does and how someone can inspect it. Markdown uses # for
見出し - リストと〔ラベル〕(URL)のリンク。 デプロイ後に際のリポジトリとライブウェブサイトのURLを追加します。
便利なREADMEは計された特かプロジェクトを含む現在の態を正直に述べます。

```
# Alex's Portfolio
```
Web開を習しながら構築されたレスポンシブな個人ポトフォリオ。

```
## Features
```
レスポンシブプロジェクトグリッド

保存された光と暗いテマ

モバイルナビゲションとプロジェクトフィルタ

```
## Built with
```
HTML、CSS、JavaScript。 ビルドステップはありません。

```
## Run locally
```
このリポジトリをダウンロドしてクロンし、index.htmlを開きます。

```
## Live website
```
公開後にGitHubページのURLを追加します。

完成したサイトのリポジトリ構造

GitHubインタフェイスのスクリンショットではなく、ファイルのツリのをえます。

ポトフォリオ

```
index.html
style.css
script.js
```

アセット/

ワクスペス.svg

プロジェクト-1.svg

プロジェクト-2.svg

プロジェクト-3.svg

```
README.md
.gitignore
```

このフォルダを公開する

選したルトでindex.htmlを保持します。

ページのメインと/(root) を使用します。

コスの束を別に保って下さい

コス全のZIPをアップロドしないでください。

```
.git is local history managed by Git;
```
手動でアップロドするフォルダではありません。

10. 選したソスルトのindex.htmlで、ポトフォリオフォルダの容を公開します。

ミニプロジェクトプロフェッショナルリポジトリ

リポジトリに明確な明、整理されたファイル名、README を指定します。 スクリンショットを追加: アセット/preview.pngで保存し、それを考にしてください![ポトフォリオスクリンショット](assets/preview.png)。 この例のアセットにはスクリンショットが含まれていません。Markdownを追加する前に、自分でキャプチャしてください。

README と .gitignore が最初の 1 に含まれていない場合は、2 番目のコミットを行います。

更をプッシュし、オンラインで確認します。 新規の訪問者がプロジェクトを理解し、ロカルで行し、その主な機能を特定し、モジュル6が完了したらライブURLを見つけることができるときにパスします。 意味のある史は、際の化を明します。何十もの空のコミットを製造しないでください。

モジュール6 ウェブサイトの展開

Goal: 的なサイトを公開し、際の更新を確認します。 ガイド時間:約15分、展開待ち時間を除く。 GitHub インタフェイスラベルは更できます。必要にじてリソスディレクトリにある公式リンクを使用します。

Lesson 6.1 デプロイとは

デプロイメントは、ホスティングサビスからファイルを利用できるようにします。 ブラウザでは、URL をリクエストし、ホストはファイルを提供し、ブラウザはペジをレンダリングします。 Git は履を保存します。Web サイトのホスト自ではありません。 このコスワクフロでは、設定された GitHub ブランチへのプッシュが Pages ビルドとデプロイメントを開始します。

地域更から公共ホムペジへ

GitHub Pages は、成功したデプロイ後に結果を表示します。

1 編集とテスト

パソコン

2 コミットとプッシュ

Git → GitHub から

3 ビルドとデプロイ

GitHub ペジ

4 訪問と確認

ブラウザは、パブリックHTTPS URLを要求します。

新しいコンテンツを確認し、コントロルをテストします。

成功をプッシュしない

展開成功を明して下さい。

11. プッシュは公開ワクフロを開始します。展開が成功した後に公開ペジを確認します。

私たちのサイトは的HTML、CSS、JavaScriptです。 GitHub Pages は、アプリケションサバなしで配信できます。 Python や PHP のバックエンドを行せず、デタベスを理したり、このプロジェクトの連絡先フォムを理します。 メトリンクは、訪問者のメルアプリに連絡先を委任します。

出版前

ロカルページを開き、名前、ページタイトル、明、連絡先の詳細を確認します。 デモメルアドレスを置き換えます。 計されたプロジェクトがに完了していると主張するクレムやリンクを削除します。 ポトフォリオプロジェクトのみをアップロドし、有料コスZIPではありません。

Lesson 6.2 GitHub ページを使って展開する

モジュル5でプッシュしたリポジトリを使用します。 以下の手順はブランチの展開と公開リポジトリを使用します。 ページの設定を管理する限が必要です。

1。 GitHubでリポジトリを開きます。 style.css と一に、リポジトリのルトにある index.html をチェックします。

```
 script.js and assets.
```

2。〔設定〕を開き、サイドバの〔ペジ〕を開きます。

3。 ビルドとデプロイメントで、ソスとしてブランチからデプロイを選します。

4。 メインと/(root)を選し、保存します。

5。 5。 ペジの展開況やリポジトリのアクションタブを調べます。 成功したビルドを待ってから、ペジの公開されたサイトリンクを使用します。

6。 新しいタブでURLを開きます。 ライブサイト上の像、ナビゲション、テマ、フィルタをテストします。

プロジェクトのサイトは、通常、URL の形をしています。

```
https://YOUR_USERNAME.github.io/devfoundry-portfolio/. Use the URL GitHub displays rather than
```

それを推測する。 You USERNAME.github.ioという名前のリポジトリは、異なるユザサイト規約であり、ここでは必要ありません。 カスタムドメインはオプションです。

チェックとトラブルシュティング

成功したプッシュは、成功した展開の明ではありません。 サイトが見つからない場合、ブランチとソスフォルダ、正確なlowercase index.htmlファイル名と最新のデプロイ結果を確認します。 スタイルや像が見つからない場合、相パスとケスを確認してください。 /assets/project-1.svg の代わりに Asset/project-1.svg を使う: 主要なスラッシュはドメインルトで始まり、リポジトリパスの下にプロジェクトサイトを破ることができます。

公式の手順:GitHub Docs、「GitHub Pagesサイトの作成」と「公開ソスの設定」。 上記のブランチワクフロをサポトし、リソスディレクトリにリンクが表示されます。

レッスン6.3 ライブウェブサイトの更新

ロカルで表示された文章を 1 つ更 セクションについて。 ロカルに保存してテストします。 更を点し、それをコミットし、それを押します。 モジュル5で最初にプッシュすると、上流が設定されますので、git push は使用できます。

```
git diff
git add index.html
git diff --cached
git commit -m "Clarify the About section"
git push
```

展開完了後、公開URLをリフレッシュし、新しい文を見つけます。 それでも古い場合は、リモトコミットが更とペジジョブが成功していることを確認します。 その後、ハドリフレッシュまたはプライベトブラウジングウィンドウをお試しください。 展開終了前のリピトはスピドアップしません。

チェックイン

ライブURL、コミットメッセジ、更された文章を、課題/test-log.md で記します。 これは、ロカルで編集、テスト、コミット、プッシュ、デプロイ、の完全なループを理解することを明します。

作業中の URL を README.md に別の通常のコミットに追加します。

レッスン6.4ドメイン

ホスティングはウェブサイトを提供しています。 ドメインは、そのホスティングを指すことができる人です。

DNSレコドは、名前を宛先に接します。 HTTPS はブラウザとホスト間のトラフィックを暗化します。 このコスを終了するためにドメインを購入する必要はありません。 GitHub PagesのURLは十分です。

ドメインを選すると、GitHubの現在のカスタムドメインの指示とレジストラの指示にってください。 DNS レコドは、 apex ドメインとサブドメインが異なるため、自の構成の設定を使用します。 その後、ドメインとHTTPSの態を確認します。

モジュル7の最終的なプロジェクト

開者ポトフォリオの構築

あなたの最終タスクは、ウェブサイトを立してパソナライズし、明することです。 あなたは、このワクブックと完全なソリュションを照することができます, しかし、コンテンツについての有意義な決定を行う, デザインと行動. 目標は、理解して維持できるサイトであり、理解せずにコピしたスクリンショットではありません。

必須コンテンツと機能

利用可能なモバイルメニュで、すべての主要なセクションに到達するナビゲション。

名前、焦点、そして明確な次の行動を持つ英雄。

セクションと本物のスキルリストについて。

タイトル、明、像、意味のあるビジュアルの代替手段を備えた少なくとも3つのプロジェクトカド。

未完成の念を明らかにします。

ワキングパブリックコンタクトメソッドと、フッタのコンタクトセクション。

デスクトップから約320pxのレスポンシブレイアウトで、水平なペジオバフロはありません。

一貫性のあるCSSスタイリング、みやすいコントラスト、表示されたキボドフォカス、ホバまたは移行フィドバック

感情の減によるサポト

プロジェクト4からのモバイルメニュインタラクションとともに、ダク/ライトのトグルおよびプロジェクトフィルタをかせて下さい。

Gitリポジトリ、便利なREADME、有意義なコミット、パブリックデプロイメント、および1つのみのライブ更新。

パソナライゼションの簡略化

アイデンティティの詳細とメタデタをすべて置換します。 自分の紹介とプロジェクトの明を書いてください。 みやすさを維持しながら、アクセントパレットやカドスタイリングなど、少なくとも2つの視的選を更します。利用可能な作業時に少なくとも1つのプロジェクト先を追加してください。 #リンクを使用して外部プロジェクトが存在しません。 短いREADMEセクションで3つのインタラクションを明します。

立したビルドシケンスを提案

紙のセクションをスケッチします。 色を選する前にコンテンツを書きみます。 HTMLのランドマクを再構築またはレビュします。 共有されたデザイントクンを適用し、レイアウトします。 より多くの動きを加える前にいビュポトをテストして下さい。 1つのインタラクションを一度にし、テストします。 リポジトリを見直し、デプロイし、新しいブラウザタブでライブURLをチェックします。

ボナス課題

最小トグルのみをしている場合は、テマ設定を保存します。 カスタムファビコンを追加します。 別のプロジェクトをビルドし、計された念を置き換えます。 感情を抑えて、優しいスクロルアニメションを追加。 プロジェクトフィルタを別のカテゴリとゼロレスメッセジで張します。 後でカスタムドメインを接します。

コンタクトフォムは、高度な張機能です。的なHTMLだけではメッセジを配信しません。 サビスまたはバックエンドを使用して、ラベルとステタスのフィドバックを提供し、サビスが確認するまでの投稿を主張することを避けます。 フロントエンドのJavaScriptでプライベトサビス資格情報を配置しないでください。

最終プロジェクト評

この rubric を使用して、作業を一貫して評します。 0から4までの各領域をスコア: 0のabsent、1質的に不完全な、2は重要なギャップできます、3は件をたします、4はそれをたし、あなたの選およびテストを記述できます。 これは、認定認ではなく、自己評ツルです。

エリア

3点の

コンテンツとHTML

必要なセクション、正直なプロジェクトの明、意味のある

見出しとユニクなID

CSSと視的な明快さ

再使用可能なスタイリング、みやすいテキスト、一貫性のあるカド、歪めなし

サイトマップ

責任ある行動

320px, 390px, 768px, 1440px のチェックパスを水平にしない

オバフロ

JavaScriptとユザビリティ

テマ、メニュ、フィルタの作業。キボドとスクリプトのフォルバックなし

チェックイン

リポジトリおよび配達

README、意味のあるコミット、ライブURL、みの更新

レコディング

タゲット: 少なくとも 15 のうち 20、すべての必須項目が完了します。 高い合計は、れた展開やアクセス不能なナビゲションを上書きしません。 欠落した本質を修正し、チェックを再行します。

あなたの評を記する

コンテンツとHTML: / 4 CSSと明快さ: / 4

答的な動作: / 4 JavaScript とユザビリティ: 4

リポジトリと配信: 4 合計: 20

どの件が最も仕事を必要としますか。 問題を書きます, あなたが作る計とそれが固定であることを明するチェックを更.

照ソリュションを見ずに明できるのは? 1つのHTMLの決定、1つのCSSルルと1つのJavaScriptの相互作用を選します。

最終プロジェクトチェックリスト

```
 [ ] My name, page title, metadata and contact details are correct.
 [ ] Navigation, hero, About, Skills, Projects, Contact and footer exist.
 [ ] At least three project cards have honest descriptions and meaningful visuals.
 [ ] Finished links reach real destinations; unfinished work is clearly labelled.
 [ ] Images use correct relative paths, dimensions and alt text.
 [ ] There is no page-level horizontal scrolling at the four test widths.
 [ ] Text remains available at 200% zoom; keyboard focus is visible.
 [ ] The mobile menu opens, closes, handles Escape and survives resizing.
 [ ] Theme state and its accessibility attribute agree; saved choice restores when available.
 [ ] Filtering shows two Web cards, one JavaScript card and all three on reset, or the
matching counts for
```

自分のカテゴリ。

```
[ ] All links and project cards remain available without JavaScript.
 [ ] Reduced-motion settings suppress unnecessary card movement.
 [ ] The console has no unexpected errors on the local or deployed page.
 [ ] My repository contains source files, assets, .gitignore and README.
 [ ] My commit history describes real changes and contains no private credentials.
 [ ] My live URL loads and I have verified a content update after deployment.
```

提出パッケージ

ライブウェブサイトのURL、GitHubリポジトリのURL、デスクトップのスクリンショット、1つ、モバイルスクリンショット1つ、更した容を明する短い段落を用意し、何が難しかったのか、次に改善するのかを明します。 完了したチェックリストを保存し、自分の習ノトでログをテストします。 他の人がサイトを開いたり、助けなしでメインパスにうことができれば、あなたは完了しました。

モジュール8 次の場所

ファイルからパブリックWebサイトへのパス全を練習しました。 次のステップは少し異なる問題で繰り返します。 フィクションサビスや小規模な情報サイト用のランディングページを作成し、同じワクフロで展開します。 コンテンツや制約の更は、自分が自に理解したアイデアを明らかにします。

的な習パス

1。 HTMLとCSSを化:フォム、アクセシビリティ、レイアウトの練習とブラウザのデバッグ。

2。 JavaScriptをより深く理解:配列、オブジェクト、、制御フロ、モジュル、エラ、非同期コド。

3。 API を使用する: データを取得し、みみと失敗の態を理します。サバにっているものを理解します。

4。 通常の JavaScript のやりとりをビルドしてデバッグできると React を探索できます。

5。 5。 プロジェクトのニズにじて、バックエンド開、デタベス、認、フルスタック展開をびます。

次のDevFoundryコスは「初心者からリアルアプリの構築まで」です。 習方向として、このワクブックは、存のフォロオン製品、リリス日、または含まれたアクセスを約束しません。

次の7つの練習セッション

セッション1:容について改善します。 セッション2: 1つのプロジェクトを追加します。 セッション3:テストキボドとズム動作。 セッション4:小さなJavaScriptウィジェットをコピせずに書きます。 セッション5:READMEを改善します。 セッション6:可視アップデトを展開し、電話で確認します。 セッション7: 明できる3つのことを書き留めて、次のトピックを勉しよう。

ウェブサイトの明

コスを完全に呼び出しる前に、コドをんだりすることなく、これらの質問に答えてください。HTMLがプロジェクトカドを記述する場所は? どのCSSルルが列のを設定しますか? テマボタンをクリックするとどうなりますか? コミット前のステジは? ペジ公開はどのブランチですか? 回答が不明な場合は、そのレッスンを見直して1つの例を追跡します。

コスのダウンロドとクイックリファレンス

ZIPは、このPDFの際の仲間です。 そのチェックポイントには、フォルダラベルを空にするだけでなく、完全なファイルが含まれています。 始動機を自分の作業フォルダにコピし、後でスナップショットを使用して差分を診します。

DevFoundry-ダウンロド/

01スタタファイル/

02-html-complete/

03-css-complete/

04-レスポンシブコンプリト/

05. javascript-complete/日本語

06最終的なプロジェクト/

課題課題

チトシト/

ビジュアル/

`README.md`

各チェックポイントには、index.html、style.css、script.js、README、バンドルされたアセットが含まれます。 アクティブCSSまたはJavaScriptファイルには、モジュルが始まるまでコメントのみが含まれています。 HTMLチェックポイントは未定です。 CSS チェックポイントは、意した固定デスクトップ幅を持っています。 レスポンシブチェックポイントは、スクリプトなしでリンクが表示されるようにします。 JavaScriptと最終チェックポイントには、すべての3つのインタラクションが含まれます。

課題は、練習の簡素化、回答ガイド、最終チェックリスト、テストログが含まれています。 cheatsheets には、次のページの 3 つの照が含まれています。 ビジュアルには、指示と照スクリンショットが含まれています。 完成したポトフォリオで使用されるすべてのコドは、付書でも印刷されます。

PDFコドでの作業

可能な限り編集可能なソスファイルを使用します。 PDFビュアは、コピとペストの間に、インデントを更したり、長い行をラップしたりすることがあります。 ラインラベル、ペジフッタ、またはソスファイルにコピしないでください。 コドを貼り付けるとき、引用符、括弧、句点およびバックティックスを保存します。 更後のブラウザコンソルを確認します。

HTMLチトシト

要素または性

使用件

```
<!DOCTYPE html>
```

現代のHTML行動を要求する

html lang="en"〉

ペジ言語の宣言

ヘッド/タイトル/メタ

メタデタ、タブのタイトルとビュポトの設定

ヘッダ/nav/メイン/フッタ

意味のあるペジ領域

セクション/記事/div

名前付きトピック、立した項目、中立ラッパ

h1 / h2 / h3 / p

コンテンツ階層と段落

ul / 李

未注文リストとリストアイテム

a href="#about" は

ユニクなid="about"にナビゲト

img src="..." alt="..."

像パスとテキストの代替

ボタン

ネイティブキボドアクセス可能なアクションコントロル

クラス/ id

再利用可能なグループ化/一意の識別子

```
hidden / data-category
```

無なコンテンツの非表示/ カスタム値の保存

チェックリスト: 有なネスティング、マッチングクロスタグ、メインランドマク、便利なテキスト、作業パス、ユニクなID。 アクションのボタンと目的地のリンクを使用します。

CSSチトシト

パタン

意味する

```
.card / #projects / h1
```

クラス/ ID/要素セレクタ

色/背景

テキストカラ/背景塗り

```
var(--accent)
```

カスタムプロパティをむ

パディング/マジン/ギャップ

空間/外空間/アイテム間の空間

ボダ/ボダラディウス

エッジ/コナラウンド

ボックスサイジング: ボダボックス

宣言された幅にパディングとボダを含める

ディスプレイ: フレックス

主要な軸線の直接子供を整理して下さい

正化コンテンツ/整列項目

主要な軸線/十字軸に合わせて下さい

フレックスラップ:ラップ

フレックスアイテムを別のラインに割りてる

ディスプレイ:グリッド

行と列レイアウトを作成する

繰り返し(3、minmax(0、1fr))

3つの均等に適用範がい格子トラック

幅:最少(1100px、calc(100% -)

2レム)

曲げられた液のページの幅

最高幅:100%;高さ:自動車

像を縮小する比例

```
@media (min-width: 640px)
```

このビュポト幅の上のルルを適用します

:hover / :フォカス可

ポインタのフィドバック/目に見えるキボドの焦点

移行/換

化を滑らかにし、要素を視的に動かして下さい

スタイルが失敗すると、マッチングセレクタ、スタイルシトパス、カスケド、スペル、欠落した括弧をチェックします。 ランダムにセレクタの特定性を加させるのではなく、要素を調べます。 最終コスのみ! 重要なルルは、レイアウトのオバライドからネイティブし性を保護します。

Gitチトシト

コマンド

それが何をするか

```
git --version
```

Gitがインストルされていることを確認してください

```
git init
```

現在のフォルダのリポジトリを作成する

```
git status
```

ステジング、ステジなし、未踏な更を表示

```
git diff
```

段落のないテキストの更を調べる

```
git add index.html
```

1つのファイルの現在の更をステジ

```
git add .
```

現在のディレクトリの下のステジ更

```
git diff --cached
```

段階的な化を見極める

```
git commit -m "Message"
```

ステジされたスナップショットをロカルに保存する

```
git log --oneline -5
```

5つの最新のコミットをレビュする

```
git remote -v
```

リモトリポジトリのアドレスを調べる

```
git push -u origin main
```

最初のプッシュと上流セットアップ

```
git push
```

設定された上流に新しいコミットを送信

```
git pull --ff-only
```

高速化可能な場合のみ、リモトの更を取得します。

力の使用か、または仕事を捨てるのではなく、史が重なり、停止し、点するので、プルが拒否した場合。 別のコンピュタまたはWebエディタでリモトが更された場合、Pullは便利です。 自分のロカルワクをコミットする代わりにはありません。 git ステタスとエラのみみは、最初の回復ツルです。

チャレンジと回答ガイド

回答をむ前に各タスクを試してみてください。 あなたの解決策は、行動を足させ、みやすいままにすると異なることができます。

チャレンジ1 コンテンツとナビゲション

見出しと段落で「今習」セクションを追加します。 ナビゲションからリンクします。 新しいリンクがそのセクションに移動すると、その ID が一意であるときにパスします。

```
Answer: create <section id="learning"> with an h2 and p inside main; add <a
href="#learning">Now
learning</a> inside #nav-links. Style it using the existing container and section classes. The
menu's
```

link-click ハンドラはにこの新しいアンカを扱います。なぜなら、ラッパをくからです。

チャレンジ2カド間隔

均等な列幅を維持しながら、すべてのカドのスペスを加させます。 カドがオバフロなしで同じグリッドにまるときを渡す。

回答: 1.4remから1.75remに.card-body Paddingを更します。 固定カドの幅は入れません。

ボダボックスは予測可能なサイジングを保ちながら、グリッドはトラック幅を制御します。

チャレンジ3 レスポンシブレイアウト

タブレットのレイアウトは640pxではなく700pxで始まります。 ナビゲションが同じブレクポイントを使用するかどうかを更する必要がありますか?

回答: プロジェクトGridの最小幅クエリを700pxに更します。 ナビゲションブレイクポイントは立しています。 それを動かすには、CSS max-width: 639pxメニュクエリとJavaScriptマッチメディアクエリを699pxに更します。 連する件を同期させ、境界幅を再テストします。

チャレンジ4 インタラクション

マッチング4枚目のカドで「その他」フィルタを追加。 マッチングカドとステタスカウントのみが表示された場合にパスします。

```
Answer: add a button with data-filter="other" and aria-pressed="false" inside #filters, then
an article
with data-category="other" inside .projects-grid. No JavaScript logic changes are required if
the
```

DOM 要素は、デフェレションされたスクリプトが行される前に存在します。 は、ハドコディングボタンの名前ではなく、カテゴリをみます。 ゼロマッチがない場合、存のカウントレポト「0件表示」; 豊富な空のメッセジはオプションの改善です。

チャレンジ5配信

小規模なコンテンツ改善を行い、それをコミットし、公開サイトの更新を明します。

回答: git diff で編集、保存、査、更されたファイルをステジ化し、git diff --cached を査し、明確なメッセジでコミットし、プッシュして展開を成功させる。 更された文章とライブURLを記します。 リポジトリのスクリンショットは、ライブページが更されたことを明しません。

トラブルシューティングデスク

私のHTMLは間違っているか、コンテンツが消えている

最初のれた領域の周りのネスティングとクロズタグをチェックします。 マクダウンフェンスをHTMLファイルに貼り付けなかったことを確認してください。 ブラウザの DOM を調べて、マクアップを解する方法を確認します。係のない作業セクションを交換するのではなく、チェックポイントで1つのセクションを比較します。

私のスタイルシトはロドしません

ファイルが style.css であることを確認します。リンクは頭にあり、 href は index.html に相的です。 ファイルを保存します。 開者ツルで失敗したリクエストを調べます。 欠落した括弧は、以下の規則に影響を及ぼす可能性があるため、別の規則を追加する前に最初の誤字ブロックを確認してください。

像はロカルで動作しますが、オンラインで失敗します

正確なファイル名と資本化を一致させます。 GitHubにコミットして表示されていることを確認してください。 コンピュタ固有のファイルパスと、このプロジェクトサイトの先頭スラッシュアセットURLを避けます。 ロカルファイルシステムが許す場合でも、公共のサイトがケスに敏感である可能性があることをえておいてください。

テマボタンやメニュが表示されない

JavaScriptで非表示にされたコントロルが公開されます。 最初のコンソルエラを探し、スクリプトパスとデフェアの性を確認し、セレクタとIDを比較します。 的および最終スクリプトの方を貼り付けた場合、コンジット宣言を重複すると、行からすべてを防ぐことができます。 ファイル全を一の完全で置換します。

テマの持性は、局所的に異なる振る舞います

file:// URL の保存ルルはブラウザによって異なります。 ロカルプレビュサバまたはデプロイされたHTTPSサイトを使用して、安定した起源にする永性を確認します。 localStorage は、オリジンとブラウザのプロファイルに特異的です。ロカルのプレビュに保存された設定は、自動的に公開ウェブサイトに送されません。

Gitは私のプッシュを拒否します

エラをおみください。 git ステタス、git リモト -v および認されたアカウントを確認します。 限エラは、履の競合とは異なります。 力プッシュで不慣れな拒絶反を解決しないでください。 このコスでは、空のリモトで始まり、一般的な別のREADMEの史の問題を避けます。

ペジは空白または404を返す

選したブランチが存在し、選したルトにindex.htmlが含まれていることを確認してください。 展開が成功し、ペジのURLを使用します。 GitHubリポジトリペジは、ライブウェブサイトと同じURLではありません。 ホストがエラを報告する場合は、ランダムなCSSを編集する前に最新のワクフロログを調べます。

リソスディレクトリ

このコスの解やはDevFoundry材です。 より深い究や、更する可能性があるセットアップやホスティング面にこれらの公式の照を使用してください。 2026年11月のサビス案が確認されました。

ツルとセットアップ

Visual Studio コドが起動: https://code.visualstudio.com/docs/getstarted/overview Git ダウンロド: https://git-scm.com/downloads GitHub Git セットアップと認: https://docs.github.com/en/get-started/git-basics/set-up-git Web プラットフォムリファレンス MDN 習ハブ: https://developer.mozilla.org/en-US/docs/docs/Learn webdocs/ja/website website/website/website/website/developers/

```
https://docs.github.com/en/pages/getting-started-with-github-pages/creating-a-github-pages-site
```

ブランチとソスフォルダの設定: https://docs.github.com/en/pages/getting-started-with-github-pages/configuring-a-publishing-a-publishing-for-your-github-pages-site カスタムドメイン:

```
https://docs.github.com/en/pages/configuring-a-custom-domain-for-your-github-pages-site
```

ビジュアルリファレンスインデックス

次のは、提供されたポトフォリオの元のやスクリンショットです。最終ポトフォリオタゲット、ブラウザファイルロル、HTMLランドマク、CSSボックスモデル、Flexbox軸、グリッドトラック、デスクトップ/タブレット/モバイルレイアウト、ライト/ダク態、Gitワクフロ、リポジトリツリ、デプロイフロ。

リポジトリツリは、GitHub の現在のインタフェイスを表示するという主張ではなく、育です。 スクリンショットは際のコスソスとバンドルされたイラストを使用します。

```
Appendix A Complete portfolio HTML
Use this complete file as a reference after Module 4. For the unstyled HTML checkpoint, leave
out
the stylesheet and script lines. For the CSS and responsive checkpoints, include the
stylesheet but
leave out the script line. The download snapshots already apply these differences.
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <meta name="description" content="Alex Morgan's web development portfolio.">
  <title>Alex Morgan | Web Developer</title>
  <link rel="stylesheet" href="style.css">
  <script src="script.js" defer></script>
</head>
<body>
  <a class="skip-link" href="#main">Skip to content</a>
  <header class="site-header">
    <nav class="container nav" aria-label="Main navigation">
      <a class="brand" href="#home">alex<span>.dev</span></a>
      <button id="menu-button" class="menu-button" hidden
        aria-controls="nav-links" aria-expanded="false">Menu</button>
      <div id="nav-links" class="nav-links">
        <a href="#about">About</a>
        <a href="#skills">Skills</a>
        <a href="#projects">Projects</a>
        <a href="#contact">Contact</a>
      </div>
      <button id="theme-button" hidden aria-pressed="false">Dark mode</button>
    </nav>
  </header>
  <main id="main">
    <section id="home" class="hero container">
      <div>
        <p class="eyebrow">LEARNING. BUILDING. SHARING.</p>
        <h1>Hi, I'm Alex.<br>I build for the web.</h1>
        <p class="intro">I'm a beginner developer turning ideas into simple,
          useful websites. This is where I share what I'm learning.</p>
        <a class="button" href="#projects">Explore my projects</a>
      </div>
      <img class="hero-art" src="assets/workspace.svg" width="520"
        height="360"
    alt="Illustrated code editor with HTML, CSS and JavaScript files">
    </section>
    <section id="about" class="container section">
      <p class="eyebrow">01 / ABOUT</p>
      <h2>Curious by default.</h2>
      <p>I enjoy solving small problems with code. I'm currently practicing
        accessible HTML, responsive CSS and interactive JavaScript.</p>
    </section>
    <section id="skills" class="container section">
      <p class="eyebrow">02 / SKILLS</p>
      <h2>My growing toolkit</h2>
      <ul class="skills">
```

```
      <li>HTML</li><li>CSS</li><li>JavaScript</li><li>Git &amp; GitHub</li>
    </ul>
  </section>
  <section id="projects" class="container section">
    <p class="eyebrow">03 / PROJECTS</p>
    <h2>Small projects. Real progress.</h2>
    <div id="filters" class="filters" hidden
      role="group" aria-label="Filter projects">
      <button data-filter="all" aria-pressed="true">All</button>
      <button data-filter="web" aria-pressed="false">Web</button>
      <button data-filter="javascript" aria-pressed="false">JavaScript</button>
    </div>
    <p id="project-count" class="muted" role="status" hidden></p>
    <div class="projects-grid">
      <article class="project-card" data-category="web">
        <img src="assets/project-1.svg" width="600" height="320"
          alt="Portfolio concept with a hero section and three project cards"
  loading="lazy">
        <div class="card-body">
          <p class="eyebrow">HTML / CSS</p>
          <h3>Personal portfolio</h3>
          <p>A home for my projects, skills and learning journey.</p>
          <a href="#home">View this portfolio</a>
        </div>
      </article>
      <article class="project-card" data-category="web">
        <img src="assets/project-2.svg" width="600" height="320"
          alt="Practice landing page concept with a headline and call to action"
  loading="lazy">
        <div class="card-body">
          <p class="eyebrow">PRACTICE CONCEPT</p>
          <h3>Product landing page</h3>
          <p>A planned practice project for layout and typography.</p>
          <span class="muted">Coming soon</span>
        </div>
      </article>
      <article class="project-card" data-category="javascript">
        <img src="assets/project-3.svg" width="600" height="320"
          alt="Planned task list app concept showing three example tasks"
  loading="lazy">
        <div class="card-body">
          <p class="eyebrow">PRACTICE CONCEPT</p>
          <h3>Task list app</h3>
          <p>A planned JavaScript project for practicing DOM events.</p>
          <span class="muted">Coming soon</span>
        </div>
      </article>
    </div>
  </section>
  <section id="contact" class="container section contact">
    <p class="eyebrow">04 / CONTACT</p>
    <h2>Let's build something.</h2>
    <p>Want to share feedback or talk about a project?</p>
    <a class="button" href="mailto:alex@example.com">Email Alex</a>
    <p
  class="muted">Demo address. Replace it with your own public contact
  address.</p>
```

```
    </section>
  </main>
  <footer class="container footer">
    <p>Alex Morgan. Built with HTML, CSS and JavaScript.</p>
    <a href="#home">Back to top</a>
  </footer>
</body>
</html>
```

Appendix B Complete portfolio CSS
The base desktop design ends immediately before the comment beginning “3.2 Mobile first overrides
and breakpoints”. Module 2 uses the base only. Module 3 appends the rest. The dark variables can
exist before the JavaScript toggle; the class simply will not be switched yet.

```
/* 2.2 Theme and type */
:root {
  color-scheme: light;
  --bg: #f8fafc;
  --surface: #ffffff;
  --text: #172033;
  --muted: #475569;
  --accent: #1d4ed8;
  --line: #cbd5e1;
}
body.dark {
  color-scheme: dark;
  --bg: #0b1220;
  --surface: #121e32;
  --text: #edf2fc;
  --muted: #b4c2d8;
  --accent: #93c5fd;
  --line: #334155;
}
/* 2.3 Box model and shared elements */
* { box-sizing: border-box; }
body {
  margin: 0;
  background: var(--bg);
  color: var(--text);
  font-family: Arial, Helvetica, sans-serif;
  font-size: 1rem;
  line-height: 1.65;
}
[hidden] { display: none !important; }
a { color: var(--accent); text-underline-offset: .2em; }
button, .button {
  display: inline-block;
  border: 1px solid var(--accent);
  border-radius: .6rem;
  padding: .65rem 1rem;
  font: inherit;
  font-weight: 700;
  cursor: pointer;
  background: var(--surface);
  color: var(--accent);
}
.button { background: var(--accent); color: var(--bg); text-decoration: none; }
button[aria-pressed="true"] { background: var(--accent); color: var(--bg); }
:focus-visible { outline: 3px solid var(--accent); outline-offset: 4px; }
.container { width: 1100px; margin-inline: auto; }
img { display: block; }
h1, h2, h3 { line-height: 1.15; letter-spacing: -.035em; }
h1 { font-size: 3.5rem; margin-block: .6rem 1.25rem; }
```

```
h2 { font-size: 2rem; margin-top: .5rem; }
h3 { font-size: 1.35rem; }
p { max-width: 65ch; }
.eyebrow { color: var(--accent); font-size: .75rem; font-weight: 700;
    letter-spacing: .12em; }
.muted { color: var(--muted); }
.intro { color: var(--muted); font-size: 1.15rem; margin-bottom: 1.6rem; }
.section { padding-block: 3rem; }
/* 2.4 Flexbox */
.site-header { border-bottom: 1px solid var(--line); }
.nav { display: flex; align-items: center; gap: 1.25rem; padding-block: 1.25rem; }
.brand {
  margin-right: auto;
  font-size: 1.3rem;
  font-weight: 800;
  text-decoration: none;
  color: var(--text);
}
.brand span { color: var(--accent); }
.nav-links { display: flex; gap: 1.25rem; }
.nav-links a { color: var(--text); text-decoration: none; }
.hero { display: flex; align-items: center; gap: 3rem; padding-block: 5rem; }
.hero > div { flex: 1; }
.hero-art { width: 42%; height: auto; }
.skills, .filters { display: flex; flex-wrap: wrap; gap: .75rem; }
.skills { padding: 0; list-style: none; }
.skills li {
  padding: .5rem 1rem;
  background: var(--surface);
  border: 1px solid var(--line);
  border-radius: 2rem;
}
/* 2.5 Grid */
.projects-grid { display: grid; grid-template-columns: repeat(3, minmax(0, 1fr));
    gap: 1.5rem; }
.project-card {
  overflow: hidden;
  background: var(--surface);
  border: 1px solid var(--line);
  border-radius: .9rem;
}
.project-card img { width: 100%; height: auto; }
.card-body { padding: 1.4rem; }
.card-body p { color: var(--muted); }
/* 2.6 Feedback */
.project-card { transition: transform .2s ease; }
.project-card:hover, .project-card:focus-within { transform: translateY(-4px); }
a:hover { text-decoration: underline; }
button:hover, .button:hover { filter: brightness(.95); }
.contact { border-top: 1px solid var(--line); }
.footer {
  display: flex;
  justify-content: space-between;
  gap: 1rem;
  padding-block: 2rem;
  border-top: 1px solid var(--line);
}
```

```
.skip-link {
  position: absolute;
  left: 1rem;
  top: -5rem;
  padding: .6rem;
  background: var(--surface);
  z-index: 10;
}
.skip-link:focus { top: .5rem; }
/* 3.2 Mobile first overrides and breakpoints */
.container { width: min(1100px, calc(100% - 2rem)); }
img { max-width: 100%; height: auto; }
h1 { font-size: clamp(2.25rem, 5vw, 3.5rem); }
.hero { flex-direction: column; align-items: stretch; gap: 2rem; padding-block:
    3rem; }
.hero-art { width: min(100%, 520px); align-self: center; }
.projects-grid { grid-template-columns: 1fr; }
.nav { flex-wrap: wrap; }
.nav-links { flex-wrap: wrap; }
.footer { flex-direction: column; }
@media (min-width: 640px) {
  .projects-grid { grid-template-columns: repeat(2, minmax(0, 1fr)); }
}
@media (min-width: 960px) {
  .projects-grid { grid-template-columns: repeat(3, minmax(0, 1fr)); }
  .hero { flex-direction: row; align-items: center; padding-block: 5rem; }
  .hero-art { width: 42%; }
  .footer { flex-direction: row; }
}
/* 3.4 Menu enhanced by JavaScript; links work without it */
@media (max-width: 639px) {
  .nav-links { width: 100%; flex-direction: column; order: 3; }
  .nav-links a { padding-block: .5rem; }
  .js .nav-links { display: none; }
  .js .nav-links.is-open { display: flex; }
}
@media (prefers-reduced-motion: reduce) {
  .project-card { transition: none; }
  .project-card:hover, .project-card:focus-within { transform: none; }
}
```

```
Appendix C Complete portfolio
JavaScript
Replace script.js with this whole file after completing Lesson 4.6 and Project 4. It contains
one set of
declarations and handlers for the three features. The HTML IDs, class names, data attributes
and
media-query widths must match the corresponding source files.
// 4.6 Theme: use a saved choice, otherwise use the system preference.
const themeButton = document.querySelector("#theme-button");
const systemTheme = window.matchMedia("(prefers-color-scheme: dark)");
let savedTheme = null;
try { savedTheme = localStorage.getItem("theme"); } catch {}
function setTheme(isDark) {
  document.body.classList.toggle("dark", isDark);
  themeButton.setAttribute("aria-pressed", String(isDark));
}
setTheme(savedTheme === "dark" ||
  (savedTheme !== "light" && systemTheme.matches));
themeButton.hidden = false;
themeButton.addEventListener("click", () => {
  const isDark = !document.body.classList.contains("dark");
  setTheme(isDark);
  try { localStorage.setItem("theme", isDark ? "dark" : "light"); } catch {}
});
// Project 4: mobile menu with keyboard and resize support.
const menuButton = document.querySelector("#menu-button");
const navLinks = document.querySelector("#nav-links");
const smallScreen = window.matchMedia("(max-width: 639px)");
function setMenu(isOpen) {
  navLinks.classList.toggle("is-open", isOpen);
  menuButton.setAttribute("aria-expanded", String(isOpen));
}
function syncMenu() {
  setMenu(false);
  menuButton.hidden = !smallScreen.matches;
}
menuButton.addEventListener("click", () => {
  setMenu(menuButton.getAttribute("aria-expanded") !== "true");
});
navLinks.addEventListener("click", (event) => {
  if (event.target.closest("a") && smallScreen.matches) setMenu(false);
});
document.addEventListener("keydown", (event) => {
  if (event.key === "Escape" && smallScreen.matches &&
      menuButton.getAttribute("aria-expanded") === "true") {
    setMenu(false);
    menuButton.focus();
  }
});
smallScreen.addEventListener("change", syncMenu);
document.documentElement.classList.add("js");
syncMenu();
// Project 4: project filtering is the additional interaction.
```

```
const filters = document.querySelector("#filters");
const cards = [...document.querySelectorAll(".project-card")];
const count = document.querySelector("#project-count");
function filterProjects(category) {
  let visible = 0;
  cards.forEach((card) => {
    card.hidden = category !== "all" && card.dataset.category !== category;
    if (!card.hidden) visible += 1;
  });
  filters.querySelectorAll("button").forEach((button) => {
    button.setAttribute("aria-pressed", String(button.dataset.filter ===
    category));
  });
  count.textContent = `${visible} project${visible === 1 ? "" : "s"} shown`;
}
filters.addEventListener("click", (event) => {
  const button = event.target.closest("button[data-filter]");
  if (button) filterProjects(button.dataset.filter);
});
filters.hidden = false;
count.hidden = false;
filterProjects("all");
```